



Anza Shaq

Security Assessment

May 5th, 2026 — Prepared by OtterSec

Alpha Toure

shxdow@osec.io

Thiago Tavares

thitav@osec.io

Table of Contents

Executive Summary	2
Overview	2
Key Findings	2
Scope	3
Findings	4
Vulnerabilities	5
OS-ANS-ADV-00 Unsound Send/Sync Bounds on Queue Types	6
Appendices	
Vulnerability Rating Scale	7
Procedure	8

01 — Executive Summary

Overview

Anza engaged OtterSec to assess the `shaq` library. This assessment was conducted between April 12th and April 18th, 2026. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 1 finding throughout this audit engagement.

In particular, we identified a vulnerability where unsound Send/Sync bounds on queue types lets non-thread-safe types cross thread boundaries through the queue ([OS-ANS-ADV-00](#)).

02 — Scope

The source code was delivered to us in a Git repository at <https://github.com/anza-xyz/shaq>. This audit was performed against commit [6cc1995](#).

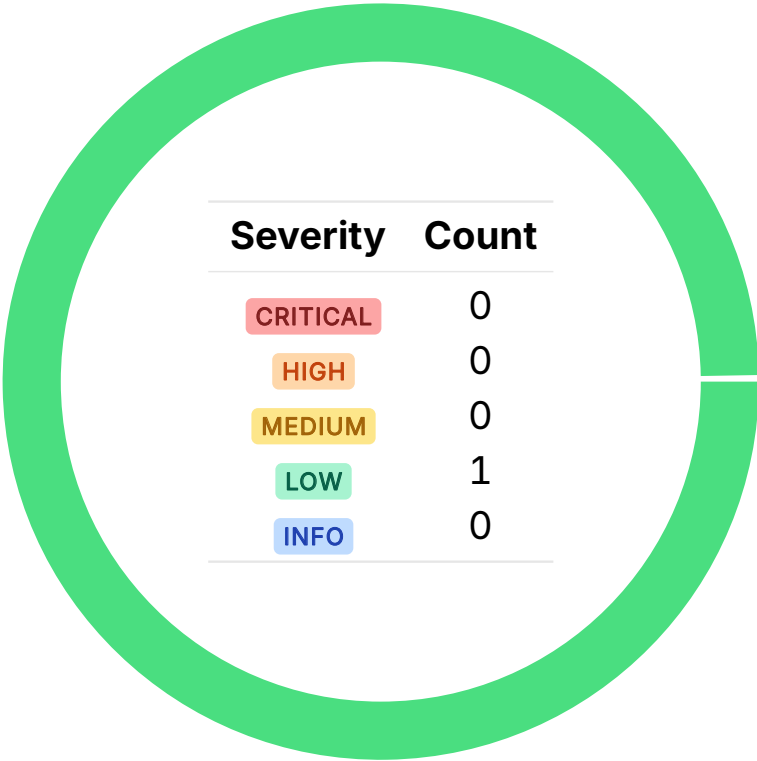
A brief description of the program is as follows:

Name	Description
shaq	A Rust library for fast shared-memory FIFO queues, letting threads or processes on the same machine exchange data through a memory-mapped ring buffer. It provides both SPSC and MPMC lock-free queue variants for low-latency communication.

03 — Findings

Overall, we reported 1 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-ANS-ADV-00	LOW	RESOLVED ✓	<code>Producer<T></code> and <code>Consumer<T></code> should not be <code>Send</code> / <code>Sync</code> for arbitrary <code>T</code> , because that lets non-thread-safe types such as <code>Rc</code> cross thread boundaries through the queue.

Unsound Send/Sync Bounds on Queue Types

LOW

OS-ANS-ADV-00

Description

In `mpmc` and `spsc`, `Producer<T>` and `Consumer<T>` are marked `Send` / `Sync` for all `T`, which is too permissive. This is unsound, since, if a user instantiates the queue with a non-`Send` type such as `Rc<_>`, the queue handles themselves may still be moved or shared across threads, which may then move or access `Rc` across thread boundaries, resulting in data races. `Sync` implementations are also affected as a type `T` is `Sync` if and only if `&T` is `Send`.

```
>_ src/mpmc.rs
```

RUST

```
unsafe impl<T> Send for Producer<T> {}  
unsafe impl<T> Sync for Producer<T> {}  
  
unsafe impl<T> Send for Consumer<T> {}  
unsafe impl<T> Sync for Consumer<T> {}
```

Remediation

Ensure to constrain `Send` implementations to the generic type `T` with the `Send` trait to prevent unsound usage. Similarly, `Sync` should require `T: Sync` (or equivalently enforce that `&T: Send`).

Patch

Resolved in [PR#69](#).

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.